

1. Introduction

Generative AI refers to AI systems that create new content such as text, images, audio, code, and synthetic data. In this project, a simple text generation model is implemented using Python and Keras. The notebook also explains how Generative Pre-trained Transformers (GPTs) work, including tokenization, transformer architecture, attention mechanisms, probability-based decoding, and ethical considerations.

```
# Environment setup
# In Google Colab, TensorFlow is usually preinstalled. If needed, uncomment the line below.
# !pip install tensorflow -q

import numpy as np
import matplotlib.pyplot as plt
import re
import random
import tensorflow as tf

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam

print('TensorFlow version:', tf.__version__)
print('Libraries imported successfully.')
```

```
TensorFlow version: 2.20.0
Libraries imported successfully.
```

2. GPT Architecture Overview

GPT models are based on the Transformer architecture. They use self-attention to understand relationships between tokens in a sequence. Text is first converted into tokens, then the model predicts the probability of the next token. GPT models are pre-trained on large text datasets and can later be fine-tuned or prompted for specific tasks. Text generation happens one token at a time by repeatedly selecting the next likely token based on the current context.

3. Dataset Preparation

A small public-domain style text dataset is used for hands-on text generation. The text resembles classic literature from sources such as Project Gutenberg. It is suitable for a simple classroom-level generative model because it contains repeated sentence structures, vocabulary patterns, and narrative style.

```
# Public-domain style training text
text_data = """
Alice was beginning to get very tired of sitting by her sister on the bank.
The sun was warm, the grass was soft, and the pages of the book seemed to dance in the light.
She wondered whether the pleasure of making a daisy chain would be worth the trouble of getting up.
Suddenly a white rabbit with pink eyes ran close by her and said that it was late.
There was nothing so very remarkable in that, but Alice followed the rabbit with curiosity.
Down went Alice after it, never once considering how in the world she was to get out again.
The passage turned and twisted, and the air grew cooler as she moved deeper into the ground.
Soon she found herself in a hall with many locked doors and a small golden key upon a glass table.
She tried the little key in every lock until one door opened into a beautiful garden.
Alice wished to enter the garden, but the doorway was too small for her to pass through.
She returned to the table and found a bottle with a paper label that said drink me.
She looked carefully to see whether it was marked poison, and finding no such word, she tasted it.
The taste was curious, like cherry tart, custard, pineapple, roast turkey, toffee, and hot buttered toast.
Soon she was small enough to go through the door, but she had forgotten the golden key.
She cried for a while, then reminded herself that crying would not solve the problem.
A curious world requires a curious mind, and Alice was determined to keep learning.
"""

# Clean text
clean_text = re.sub(r'^a-zA-Z\s', '', text_data.lower())
clean_text = re.sub(r'\s+', ' ', clean_text).strip()

print('Number of characters:', len(clean_text))
print('Preview:', clean_text[:250])
```

```
Number of characters: 1418
Preview: alice was beginning to get very tired of sitting by her sister on the bank the sun was warm the grass was soft and the pages of the book seemed to dance in
```

4. Tokenizer and Training Sequences

The tokenizer converts words into numerical IDs. A neural network cannot directly process raw text, so text must be transformed into numerical sequences before training.

```
# Tokenizer loading and fitting
# The tokenizer learns the vocabulary and converts words into integer IDs.
tokenizer = Tokenizer(oov_token='<OOV>')
tokenizer.fit_on_texts([clean_text])
```

```

total_words = len(tokenizer.word_index) + 1
print('Total vocabulary size:', total_words)
print('Example word index:', list(tokenizer.word_index.items())[:10])

# Create n-gram training sequences
input_sequences = []
words = clean_text.split()

for i in range(2, len(words)):
    sequence = words[:i+1]
    encoded = tokenizer.texts_to_sequences([' '.join(sequence)])[0]
    input_sequences.append(encoded)

max_sequence_len = max(len(seq) for seq in input_sequences)
input_sequences = pad_sequences(input_sequences, maxlen=max_sequence_len, padding='pre')

X = input_sequences[:, :-1]
y = input_sequences[:, -1]
y = tf.keras.utils.to_categorical(y, num_classes=total_words)

print('Input shape:', X.shape)
print('Output shape:', y.shape)
print('Maximum sequence length:', max_sequence_len)

```

```

Total vocabulary size: 158
Example word index: [(<'<OOV>', 1), ('the', 2), ('was', 3), ('she', 4), ('a', 5), ('to', 6), ('and', 7), ('alice', 8), ('in', 9), ('of', 10)]
Input shape: (268, 269)
Output shape: (268, 158)
Maximum sequence length: 270

```

5. Build and Train a Basic Generative Model

This model uses an embedding layer, an LSTM layer, dropout, and a softmax output layer. The softmax layer predicts the probability of the next word in the sequence.

```

# Build a simple text generation model
model = Sequential([
    Embedding(input_dim=total_words, output_dim=64, input_length=max_sequence_len-1),
    LSTM(96),
    Dropout(0.2),
    Dense(total_words, activation='softmax')
])

model.compile(
    loss='categorical_crossentropy',
    optimizer=Adam(learning_rate=0.01),
    metrics=['accuracy']
)

model.summary()

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential"

```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

```

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)

```

```

# Train the model
history = model.fit(
    X,
    y,
    epochs=6,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)

```

```

Epoch 1/6
14/14 ----- 5s 165ms/step - accuracy: 0.0467 - loss: 5.0701 - val_accuracy: 0.0556 - val_loss: 5.0928
Epoch 2/6
14/14 ----- 3s 237ms/step - accuracy: 0.0935 - loss: 4.7099 - val_accuracy: 0.0556 - val_loss: 5.4297
Epoch 3/6
14/14 ----- 2s 139ms/step - accuracy: 0.0841 - loss: 4.5391 - val_accuracy: 0.0556 - val_loss: 5.5078
Epoch 4/6
14/14 ----- 2s 137ms/step - accuracy: 0.0888 - loss: 4.2660 - val_accuracy: 0.0741 - val_loss: 5.5815
Epoch 5/6
14/14 ----- 2s 139ms/step - accuracy: 0.0935 - loss: 3.8401 - val_accuracy: 0.0926 - val_loss: 5.3953
Epoch 6/6
14/14 ----- 3s 184ms/step - accuracy: 0.1682 - loss: 3.2827 - val_accuracy: 0.0741 - val_loss: 5.9707

```

6. Training Visualizations

```
# Plot training and validation loss/accuracy
plt.figure(figsize=(7,4))
plt.plot(history.history['loss'], label='Training loss')
plt.plot(history.history['val_loss'], label='Validation loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()

plt.figure(figsize=(7,4))
plt.plot(history.history['accuracy'], label='Training accuracy')
plt.plot(history.history['val_accuracy'], label='Validation accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



7. Text Generation Function

The generation function supports output length, temperature, top-k, and top-p controls. These parameters are important because they affect creativity, randomness, and coherence of generated text.

```
def sample_with_temperature(predictions, temperature=1.0, top_k=None, top_p=None):
    """Sample the next token using temperature, top-k, and top-p nucleus sampling."""
    predictions = np.asarray(predictions).astype('float64')
    predictions = np.log(predictions + 1e-9) / temperature
    exp_preds = np.exp(predictions)
    probabilities = exp_preds / np.sum(exp_preds)

    # Apply top-k filtering
    if top_k is not None:
        top_indices = probabilities.argsort()[-top_k:]
        filtered = np.zeros_like(probabilities)
        filtered[top_indices] = probabilities[top_indices]
        probabilities = filtered / np.sum(filtered)

    # Apply top-p filtering
    if top_p is not None:
        sorted_indices = np.argsort(probabilities)[::-1]
        sorted_probs = probabilities[sorted_indices]
        cumulative_probs = np.cumsum(sorted_probs)
        cutoff = cumulative_probs <= top_p
        if not np.any(cutoff):
            cutoff[0] = True
        allowed_indices = sorted_indices[cutoff]
        filtered = np.zeros_like(probabilities)
        filtered[allowed_indices] = probabilities[allowed_indices]
        probabilities = filtered / np.sum(filtered)
```

```

return np.random.choice(len(probabilities), p=probabilities)

def generate_text(seed_text, next_words=20, temperature=1.0, top_k=None, top_p=None):
    """Generate text from a seed prompt using the trained LSTM model."""
    generated = seed_text.lower()

    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([generated])[0]
        token_list = pad_sequences([token_list], maxlen=max_sequence_len-1, padding='pre')
        predicted_probs = model.predict(token_list, verbose=0)[0]
        predicted_index = sample_with_temperature(predicted_probs, temperature, top_k, top_p)

        next_word = None
        for word, index in tokenizer.word_index.items():
            if index == predicted_index:
                next_word = word
                break

        if next_word is None or next_word == '<OOV>':
            continue

        generated += ' ' + next_word

    return generated

print('Text generation function is ready.')

```

Text generation function is ready.

Exercise 1: Basic Text Generation

```

seed = 'alice was'
print(generate_text(seed, next_words=25, temperature=0.8))

```

alice was to to soft the to in she through she soft she get world in the tasted of after it after out on the garden and

Exercise 2: Controlling Output Length

```

# Exercise 2: Controlling Output Length

```

```

print("Short output:")
print(generate_text(
    "the garden",
    next_words=10,
    temperature=0.8
))

print("\nLonger output:")
print(generate_text(
    "the garden",
    next_words=30,
    temperature=0.8
))

```

Short output:
the garden to pass very of considering hot and the very to

Longer output:
the garden to custard to to nothing as it the making found that a rabbit pink locked getting there by up went ran by once had the how as the there the

Exercise 3: Adjusting Temperature

Lower temperature usually gives safer, more predictable text. Higher temperature can create more diverse but less stable text.

```

# Exercise 3: Adjusting Temperature

```

```

print("Temperature 0.5:")
print(generate_text(
    "alice found",
    next_words=20,
    temperature=0.5
))

print("\nTemperature 1.2:")
print(generate_text(
    "alice found",
    next_words=20,
    temperature=1.2
))

```

Temperature 0.5:
alice found to beginning to to the cherry custard was cherry cherry tart custard to to to small whether to to said

Temperature 1.2:
alice found beginning to making the table and door so by there by looked eyes beginning a her and solve very that

Exercise 4: Using Top-k and Top-p

Top-k limits generation to the k most likely next tokens. Top-p nucleus sampling selects from the smallest group of tokens whose cumulative probability reaches p.

```
# Exercise 4: Using Top-k and Top-p
```

```
print("Top-k generation:")
print(generate_text(
    "the rabbit",
    next_words=20,
    temperature=0.8,
    top_k=5
))

print("\nTop-p generation:")
print(generate_text(
    "the rabbit",
    next_words=20,
    temperature=0.8,
    top_p=0.8
))
```

Top-k generation:
the rabbit to through to to very tired and by her and the on the bank the bank she found and in

Top-p generation:
the rabbit she to beginning to get as to in the door said sitting she after the curious cherry custard tart cherry

Exercise 5: Prompt Engineering

Prompt engineering means changing the input prompt to guide the style, topic, or structure of the generated output.

```
# Exercise 5: Prompt Engineering
```

```
prompts = [
    "write a short story about alice",
    "describe a mysterious garden",
    "continue this sentence the golden key"
]

for prompt in prompts:
    print("\nPrompt:", prompt)
    print(
        generate_text(
            prompt,
            next_words=25,
            temperature=0.9,
            top_k=8
        )
    )
```

Prompt: write a short story about alice
write a short story about alice very tired and sitting alice sitting of by up sitting with sitting with up up white suddenly a rabbit with with with curiosity alice

Prompt: describe a mysterious garden
describe a mysterious garden she through she through the warm was warm the found she found in and and the tried the tried and and and a glass small

Prompt: continue this sentence the golden key
continue this sentence the golden key get very by her of of by her her alice ran the beginning of by by sitting by her that by up rabbit of by

8. Practical Application Demonstration

A practical use of this model is simple content creation, such as generating first-draft story ideas, writing prompts, or marketing captions.

In a real application, the model could be integrated into a writing assistant where users provide a seed phrase and choose creativity settings such as output length and temperature.

9. Ethical Considerations

Generative AI can produce biased, false, harmful, or copyrighted-looking content if it is trained on poor-quality data or used without oversight. Ethical controls should include dataset review, human approval for sensitive uses, transparency that content is AI-generated, bias monitoring, privacy protection, and clear usage policies.

10. Conclusion

This project demonstrated the basics of Generative AI by explaining GPT concepts and implementing a simple text generation model. The notebook covered environment setup, tokenizer and model creation, text generation, output length control, temperature, top-k, top-p, prompt engineering, applications, and ethical considerations.

